Ingegneria del Software — Soluzione del Tema 18/01/2021

Esercizio 1

Per incentivare l'uso di carte e app di pagamento, il governo ha definito un piano di rimborso per gli utenti che ne fanno uso. Un utente ha diritto a un rimborso normale (normal cashback) se effettua almeno 50 transazioni: il rimborso è del 10% delle spese effettuate fino a un massimo di 150 euro. I 100000 utenti che effettuano più transazioni hanno anche diritto a un rimborso super (super cashback) di 1500 euro. In caso di utenti con lo stesso numero di transazioni, si privilegiano gli utenti per cui la prima transazione effettuata è più vecchia. Si assume che due transazioni non possono mai essere perfettamente contemporanee.

Si consideri la classe `TransactionsDB` che definisce un database di transazioni ai fini di monitorare il piano di rimborsi. Il database utilizza la classe pura `Transaction` di cui sono riportati i metodi più rilevanti.

```
public class TransactionsDB {
  //@ensures (* Aggiunge la transazione, lanciando una NullPointerException se e' nulla *)
  public void addTransaction(Transaction t);

  //@ensures (* \result e' la lista di transazioni effettuate da uId in ordine cronologico *)
  public /*@ pure @*/ List<Transaction> userTransactions(UserId uId);

  //@ensures (* \result == ammontare del rimborso normale per uId *)
  public /*@ pure @*/ float normalCashBackFor(UserId uId);

  //@ensures (* \result == ammontare del rimborso super per uId *)
  public /*@ pure @*/ float superCashBackFor(UserId uId);
}

public /*@ pure @*/ class Transaction {
  //@ensures (* \result == utente che ha effettuato la transazione *)
  public UserId getUserId();

  //@ensures (* \result == ammontare della transazione *)
  public float getAmount();

  //@ensures (* \result <==> la transazione e' precedente a other  *)
  public boolean before(Transaction other);
}
```

Domanda a)

Si specifichino in JML i metodi `normalCashBackFor()` e `superCashBackFor()`.

Soluzione

```
//@requires uId != null
//@ensures userTransactions(uId).size() < 50 ==> \result == 0 &&
//@userTransactions(uId).size() >= 50 ==> \result ==
//@  (\sumof int i; i>=0 && i<userTransactions(uId).size();
//@    userTransactions(uId).get(i).getAmount()) > 1500 ? 150 :
//@  (\sumof int i; i>=0 && i<userTransactions(uId).size();
//@    userTransactions(uId).get(i).getAmount()) / 10;
public /*@ pure @*/ float normalCashBackFor(UserId uId);

//@requires uId != null
//@ensures \result ==
//@ (\numof UserId other; other!=null && !other.equals(uId);
//@    (userTransactions(other).size()>userTransactions(uId).size())) +
//@ (\numof UserId other; other!=null && !other.equals(uId);
//@    userTransactions(other).size()==userTransactions(uId).size() &&
//@    !userTransactions(other).isEmpty() &&
```

```
//@    userTransactions(other).get(0).before(userTransactions(uId).get(0)))
//@ < 100000 ? 1500 : 0;
public /*@ pure @*/ float superCashBack(UserId uId);
```

Domanda b)

Si consideri un'implementazione che utilizza una lista per contenere le transazioni degli utenti, come mostrato di seguito.

```
public class TransactionsDB {
  private final List<Transaction> list;
  ...
}
```

Per tale implementazione, si definiscano l'invariante di rappresentazione e la corrispondente funzione di astrazione.

Soluzione

Nell'invariante di rappresentazione escludiamo che la lista sia nulla, escludiamo possibili valori nulli nella lista ed escludiamo valori duplicati.

```
//@(* Representation invariant RI *)
//@private invariant list!=null &&
//@(\forall int i; i>=0 && i<list.size(); list.get(i)!=null &&
//@    (\forall int j; j>=0 && j<i; !list.get(j).equals(list.get(i)) ) )
```

La funzione di astrazione definisce `userTransactions()` in funzione della lista usata nella rappresentazione. Gli altri metodi pubblici sono infatti definiti a partire da questo metodo. Si assume che il fatto che `userTransactions()` ritorni le transazioni in ordine cronologico sia definito come postcondizione del metodo.

```
//@(* Abstraction function AF *)
//@private invariant
//@(\forall Transaction t; t!=null;
//@    userTransactions(t.getUserId()).contains(t) <==>
//@    (\exists int i; i>=0 && i<list.size(); list.get(i).equals(t)) )
```

Domanda c)

Si consideri ora una classe `TransactionsDB2` che aggiunge un metodo `totalCashBackFor(UserId uid)` che ritorna il cash back totale a cui ha diritto `uId`, come somma di cash back normale e super. È possibile definire `TransactionsDB2` come sottoclasse di `TransactionsDB` in accordo con il principio di sostituzione? È possibile il viceversa?

Soluzione

Trattandosi di un'estensione pura, che aggiunge solo un metodo e non cambia le proprietà, è possibile definire `TransactionsDB2` come sottoclasse di `TransactionsDB`.

Il viceversa non sarebbe possibile, in quanto non si possono rimuovere metodi nelle sottoclassi (regola della segnatura in Java).

Esercizio 2

Si consideri la seguente classe Java:

```
public class Vector {
  private double x;
  private double y;
  public Vector(double x, double y) {
    this.x = x; this.y = y;
  }
  public double getX() { return x; }
  public double getY() { return y; }
  public Vector sum(Vector other) {
    Vector res =  new Vector(this.x+other.x, this.y+other.y);
    return res;
  }
}
```

Domanda a)

E' possibile invocare i metodi della classe da thread separati senza che questo crei problemi? In caso affermativo motivare la risposta, in caso negativo indicare come andrebbe modificato il codice per avere una sincronizzazione corretta.

Soluzione

La classe rappresenta un oggetto immutabile. Come tale non presenta problemi di sincronizzazione, anche a fronte dell'invocazione dei suoi metodi da parte di thread concorrenti.

Domanda b)

Si aggiunga alla classe il metodo:

```
public void set(double x, double y)
```

che imposta il valore delle due componenti `x` e `y` del vettore. Si scriva il codice del metodo (e se necessario si cambi il codice degli altri metodi) in maniera che non si presentino problemi di concorrenza.

Soluzione

Il metodo `set` rende l'oggetto mutabile. Il metodo dovrà quindi essere `synchronized` e andranno sincronizzati anche i due getter. Più delicata la sincronizzazione del metodo `sum` che rischia di generare un deadlock. Si riporta il codice dei due metodi `set` e `sum`:

```
public synchronized void set(double x, double y) {
  this.x = x; this.y = y;
}

public Vector sum(Vector other) {
  double otherX, otherY;
  synchronized(other) {
    otherX = other.x;
    otherY = other.y;
  }
  Vector res;
  synchronized(this) {
    res =  new Vector(this.x+other.x, this.y+other.y);
  }
  return res;
}
```

In alternativa il metodo `sum` poteva essere riscritto in modo più semplice come segue:

```
public Vector sum(Vector other) {
  return new Vector(this.getX()+other.getX(), this.getY()+other.getY());
}
```

eliminando corse critiche e rischio di deadlock a fronte di un comportamento che non garantisce atomicità: tra due invocazioni di `getX` e `getY` potrebbe essere invocato il metodo `set`.

Domanda c)

Si aggiunga alla classe il metodo:

```
public void suspendIfDifferent()
```

che sospende il thread chiamante in attesa che le componenti `x` e `y` del vettore diventino uguali. Si spieghi se e come vanno cambiati gli altri metodi.

Soluzione

Il metodo `suspendIfDifferent` può essere scritto come segue:

```
public synchronized void suspendIfDifferent() throws InterruptedException {
  while(x!=y) wait();
}
```

inoltre va inserita l'istruzione `if(x==y) notifyAll();` all'interno del metodo `set`.

Esercizio 3

Si considerino le seguenti classi Java:

```
abstract class Vehicle {
  public void print(Vehicle other) {
    System.out.println("I am a "+who()+" other is a "+other.who());
  }
  public abstract String who() ;
}

class Car extends Vehicle {
  public void print(Car other) {
    System.out.println("Someone says I am a "+who()+", other is a "+other.who());
  }
  public String who() { return "car"; }
}

class ElectricCar extends Car {
  public void print(Car other) {
    System.out.println("I am sure I am a "+who()+", other is a "+other.who());
  }
  public String who() { return "electric car"; }
}

class Bike extends Vehicle {
  public void print(Vehicle other) {
    System.out.println("I believe I am a "+who()+", other is a "+other.who());
  }
  public String who() { return "bike"; }
}
```

Si consideri poi il seguente frammento di codice Java che utilizza le classi dichiarate sopra:

```
1.    Vehicle v1 = new Vehicle();
2.    Vehicle v2 = new Car();
3.    Vehicle v3 = new ElectricCar();
4.    Vehicle v4 = new Bike();
5.    Car c = new Car();
6.    ElectricCar ec1 = new Car();
7.    ElectricCar ec2 = new ElectricCar();
8.    v1.print(ec1);
9.    v2.print(v3);
10.   v3.print(v4);
11.   v3.print(c);
12.   c.print(v3);
13.   c.print(ec2);
14.   ec2.print(c);
```

Domanda a)

Si indichino quali righe del codice sopra riportato generano un errore in fase di compilazione, motivando brevemente la propria risposta.

Soluzione

- Riga 1. Non è possibile instanziare una classe astratta (`Vehicle`)
- Riga 6. La classe `Car` non è sottoclasse di `ElectricCar` ma sopraclasse. Non è possible assegnare una `Car` and una `ElectricCar`.
- Riga 8. A seguito del primo errore `v1` non è definito.

Domanda b)

Supponendo che tutte le righe che provocano errori siano state rimosse, scrivere l'output prodotto dal programma, motivando brevemente la risposta.

Soluzione

```
9.  I am a car other is a electric car
10. I am a electric car other is a bike
11. I am a electric car other is a car
12. I am a car other is a electric car
13. Someone says I am a car, other is a electric car
14. I am sure I am a electric car, other is a car
```

Esercizio 4

Si consideri il seguente metodo statico:

```
1  public static int  foolCode(int n, int x) {
2     if (n > 0 && x < 100) {
3        int j = 0;
4        while (2 * j < n) {
5           j++;
6           n--;
7        }
8     }
9     return n*x;
10 }
```

Domanda a)

Si definisca un insieme minimo di casi di test che copra tutte le istruzioni.

Soluzione

Un caso basta, p.es. `n=1, x=1`.

Domanda b)

Si definisca un insieme minimo di casi di test che copra tutte le diramazioni (decisions – branch coverage).

Soluzione

Oltre al caso precedente anche p.es. `n=0,x=1`.

Domanda c)

Si definisca un insieme minimo di casi di test che copra tutte le diramazioni e tutte le condizioni (decisions and conditions coverage).

Soluzione

Oltre ai casi precedenti anche p.es. `n=1,x=100`.

Domanda d)

Si sintetizzi un caso di test per attraversare il cammino : 1 2 3 4 5 6 7 4 5 6 7 8 9 (che quindi attraversa il ciclo due volte).

Soluzione `n=4, x=1`